Drug Shortage Prediction System

Client Manual and Deployment Guide

Prepared For:	Client
Prepared By:	Capstone Team
Version:	Version 2.0
Date:	March 24, 2026
Source of Truth:	`CapstoneUC2_Source_Code.ipynb`

This manual documents the drug shortage prediction solution implemented in `CapstoneUC2_Source_Code.ipynb`. It is written for client stakeholders, technical teams, and future maintainers who need a clear understanding of the business purpose, data foundations, model workflow, deployment steps, dashboard outputs, and interpretability artifacts produced by the current implementation.

Contents

Document Note 1

I Business Overview 2

1 How to Read This Manual 3
1.1 Purpose . . . 3
1.2 Audience . . . 3
1.3 How the Document Is Organized . . . 3

2 Executive Summary 4
2.1 What the System Does . . . 4
2.2 Why It Matters . . . 4
2.3 Current Recorded Final Run . . . 5

3 System at a Glance 6
3.1 End-to-End Workflow . . . 6
3.2 Pipeline Module Map . . . 6
3.3 Core Modeling Grain . . . 7

II Technical Workflow 8

4 Business Problem and Operating Use 9
4.1 Business Question . . . 9
4.2 How the Output Should Be Used . . . 9
4.3 How the Output Should Not Be Used . . . 9

5 Data Foundation and External Data Integration 10
5.1 Input Data Categories . . . 10
5.2 Core Input Datasets . . . 10
5.3 External and Reference Data Sources . . . 11
5.4 How External Data Is Integrated . . . 12
5.4.1 DPD . . . 12
5.4.2 Google Trends . . . 12
5.4.3 CPI . . . 12
5.4.4 FluNet . . . 12
5.5 DPD Contribution in the Recorded Run . . . 12
5.5.1 Molecule Source Breakdown . . . 12
5.5.2 ATC Source Breakdown . . . 13
5.6 Key Business Definitions . . . 13
5.7 Data Standardization Rules . . . 13

6 Label Definition, Time Alignment, and Features **14**
6.1 Prediction Target . . . 14
6.2 Label Definition . . . 14
6.3 Prediction Timing Assumption . . . 14
6.4 Feature Engineering Summary . . . 14
6.5 Feature Families . . . 15
6.6 Most Important Features in the Recorded Final Run . . . 15

7 Modeling Workflow **17**
7.1 Temporal Split . . . 17
7.2 Benchmarks and Candidate Models . . . 17
7.2.1 Benchmark Models . . . 17
7.2.2 Tuned Candidate Models . . . 17
7.3 Hyperparameter Tuning . . . 18
7.4 Threshold Optimization . . . 18
7.5 Automated Model Selection Logic . . . 18
7.6 Recorded Final Test Performance . . . 18

8 Interpretability and Explanation Outputs **21**
8.1 Interpretability Layers . . . 21
8.2 Validation-Stage Interpretability . . . 21
8.3 Test-Stage Interpretability . . . 21
8.4 Row-Level Explanation Export . . . 24

9 Deployment and Azure SQL Export **25**
9.1 Deployment Target . . . 25
9.2 Runtime and Environment Requirements . . . 25
9.3 Execution Order . . . 25
9.4 Default Deployment Logic . . . 26
9.5 Export Behavior . . . 26
9.6 Output Tables . . . 26
9.7 What the Client Uses Most Often . . . 27

III Client Guidance and Reference **28**

10 How to Use the Results **29**
10.1 How to Read the Risk Score . . . 29
10.2 How to Read the Risk Bucket . . . 29
10.3 How to Read Predicted and Actual Shortage . . . 29
10.4 How to Use Performance Tables . . . 29
10.5 How to Use Reasons . . . 30

11 Risks, Assumptions, and Limitations **31**
11.1 Operational Assumptions . . . 31
11.2 Key Risks . . . 31
11.3 Known Limitations . . . 31

A Appendix A: Azure SQL Output Dictionary **32**
A.1 dbo.dim_time 32
A.2 dbo.dim_molecule 32
A.3 dbo.dim_item 32
A.4 dbo.model_registry 33
A.5 dbo.run_registry 33
A.6 dbo.fact_risk_score 34
A.7 dbo.fact_risk_reason 34
A.8 dbo.model_performance_summary 35
A.9 dbo.model_performance_weekly 35

B Appendix B: Additional Test Diagnostics **37**

C Appendix C: Glossary **39**

Document Note

This manual is based only on `CapstoneUC2_Source_Code.ipynb`. Other notebooks, experiments, and intermediate scripts were intentionally excluded so that the client-facing documentation remains aligned to a single authoritative implementation.

The current implementation exports dashboard outputs to Azure SQL under the `dbo` schema. The notebook also resolves its base path from the current working directory. As a result, the working directory used to run the notebook must contain the `Internaldata` folder.

Part I

Business Overview

Chapter 1

How to Read This Manual

1.1 Purpose

This manual has two goals:

- explain the system clearly for client stakeholders.
- provide enough operational detail for deployment, maintenance, and dashboard use.

1.2 Audience

This document is intended for:

- business stakeholders,
- analytics and reporting teams,
- technical client teams responsible for deployment,
- future maintainers of the notebook and Azure SQL outputs.

1.3 How the Document Is Organized

The document is organized in three layers:

1. **Business overview**: what the system does and why it matters.
2. **Technical workflow**: data, labels, feature logic, model workflow, interpretability, and deployment.
3. **Reference**: Azure SQL output dictionaries and glossary.

Key Takeaway

A business reader can focus on Chapters 2 and 3. A technical reader responsible for deployment or maintenance should focus on Chapters 4 through 8.

Chapter 2

Executive Summary

2.1 What the System Does

The system predicts short-term drug shortage risk at the **item-week** level. For each operational item and week, it estimates whether the item is likely to experience a shortage within the next four weeks.

The solution combines:

- internal operational history,
- item and product master data,
- vendor concentration and lead-time features,
- weekly sales information,
- external contextual signals,
- and standardized reference enrichment.

2.2 Why It Matters

The business value of the system is early warning. Rather than waiting for a shortage to become fully visible in downstream reporting, the client receives a forward-looking risk signal that can support:

- earlier monitoring,
- triage of high-risk items,
- targeted operational review,
- faster response planning,
- and clearer performance monitoring through Azure SQL dashboard tables.

2.3 Current Recorded Final Run

The notebook uses **automated model selection**. In the current implementation, the final model is selected from the tuned tree-based candidate set after hyperparameter tuning and threshold optimization.

In the current recorded run:

- the selected final model is `LIGHTGBM`,
- the selected threshold is `0.4543`,
- the development-stage model score from Section 7 is `0.6139`,
- the final held-out test PR-AUC is `0.5869`,
- the final held-out test precision, recall, and F1 are `0.5139`, `0.5915`, and `0.5500`.

Item	Recorded Value
Prediction target	`label_4w`
Engineered feature columns	121
Final modeling features	99
Categorical features	8
Numerical features	91
Training rows	141,705
Validation rows	28,341
Test rows	37,788
Primary model selection anchor	Highest Section 7 rolling-fold mean PR-AUC across tuned candidates, which keeps the selection rule aligned to forward-looking ranking quality rather than a single thresholded metric.
Secondary tie-break	Prefer higher validation recall when validation F1 is within 0.05 of the current best candidate, which gives the deployment logic a slight bias toward finding more true shortage cases when models perform similarly overall.
Selected model in current run	`LIGHTGBM`
Selected threshold in current run	0.4543
Section 7 model score	0.6139
Validation precision	0.5085
Validation recall	0.6731
Validation F1	0.5793
Final test PR-AUC	0.5869
Final test precision	0.5139
Final test recall	0.5915
Final test F1	0.5500

Key Takeaway

The correct client-facing statement is: the notebook implements automated model selection, and in the current recorded run that logic selected `LIGHTGBM` with a final deployment threshold of `0.4543`.

Chapter 3

System at a Glance

3.1 End-to-End Workflow

At a high level, the notebook performs the following workflow:

1. acquire and refresh external data,
2. load and standardize internal data,
3. build the item universe and weekly modeling panel,
4. engineer leakage-safe predictive features,
5. define the forward-looking 4-week shortage label,
6. split the data into train, validation, and test periods,
7. benchmark baseline and candidate models,
8. tune model hyperparameters on rolling folds inside the training window,
9. optimize decision thresholds on validation,
10. select the final model,
11. evaluate final out-of-sample performance on test,
12. export scores, performance tables, and reasons to Azure SQL.

3.2 Pipeline Module Map

The table below summarizes how the major notebook stages map to business and technical pipeline responsibilities.

Module	Notebook Stage	Role in the Pipeline
External acquisition	Section 1	Download and refresh DPD, Google Trends, CPI, and FluNet artifacts so the model has the latest approved contextual inputs available to the current run.
Data foundation	Section 2	Build the clean item scope, standardized identity layer, and aligned weekly data foundation used by every downstream join and export.
Feature layer	Section 3	Create the model-ready item-week panel and build the forward-looking target in a leakage-safe manner.
Modeling preparation	Section 4	Create leakage-safe train, validation, and test splits and validate matrix integrity before any candidate model is compared.
Benchmarking	Sections 5 and 6	Compare baselines and untuned candidate models to show whether more complex models are adding real value.
Optimization	Sections 7 and 8	Tune candidate models and optimize operational thresholds for the final deployment setting.
Final selection	Section 9	Select the final model and generate validation-stage interpretability outputs.
Final test reporting	Section 10	Produce held-out test performance outputs and test-stage SHAP artifacts.
Deployment	Section 11	Export dimensions, registries, scores, reasons, and performance-monitoring tables to Azure SQL.

3.3 Core Modeling Grain

The main modeling key is:

$$(\texttt{ITEM_NUM_CLEAN}, \texttt{date_monday})$$

The principal business identifiers used throughout the solution are:

- `ITEM_NUM_CLEAN`,
- `DIN_KEY`,
- `FINAL_MOLECULE`,
- `ATC_LEVEL1`,
- `ATC_LEVEL3`,
- `VENDOR_NUM`.

Key Takeaway

The solution is not forecasting at the patient level, prescription level, or aggregate product-family level. Its operating unit is one item in one week.

Part II

Technical Workflow

Chapter 4

Business Problem and Operating Use

4.1 Business Question

The core business question is:

> Given the information available at the start of week t, which items are at elevated risk of entering shortage during weeks $t + 1$ through $t + 4$?

4.2 How the Output Should Be Used

The intended use case is prioritization and triage. The risk signal is designed to help the client:

- identify which items require attention first,
- support earlier operational review,
- monitor class-level or molecule-level risk patterns,
- and compare predicted versus observed shortage behavior over time.

4.3 How the Output Should Not Be Used

The system is not intended to:

- replace operational judgment,
- prove causal drivers of shortage,
- or serve as a stand-alone decision engine without human review.

Chapter 5

Data Foundation and External Data Integration

5.1 Input Data Categories

The notebook uses three broad categories of input data:

- **Internal operational data**: item master data, purchase orders, receptions, lead times, and weekly operational history.
- **External contextual data**: market, macroeconomic, and public-health signals.
- **Reference enrichment data**: official drug and therapeutic mappings used to standardize identity and classification.

5.2 Core Input Datasets

The following datasets form the core inputs to the current implementation. Together they provide the operational, reference, and contextual signals used throughout the workflow.

Dataset	Type	Grain	Primary Role
activeitems.csv	Internal	Item	Product master and handling attributes used to define the operational item universe and product display fields.
train_P2_inbound_allitems_3years.csv	Internal	Item-week	Core operational and shortage-history panel that anchors the supervised learning dataset.
promitto_purchase_orders(2).csv	Internal	Transaction	Ordering, vendor, and purchasing behavior used to create supply-side and concentration features.
promitto_receptions(2).csv	Internal	Transaction	Received quantity and fulfillment history used to reflect actual inbound performance and short-term supply friction.
networkleadtimes.csv	Internal	Vendor / route	Lead-time exposure inputs used to capture logistics latency and replenishment risk.
promittoactivedcs.csv	Internal	Reference	Distribution-center support context used to enrich operational routing and supply structure.
promitto_weekly_sales.csv	Internal	Item-week	Weekly sales and fulfillment-related demand features used to measure recent demand volume and volatility.
ATC_COMPOSITE_TRENDS.csv	External	ATC-week	Therapeutic trend intensity used as a market-attention or demand-context signal after time-safe lagging.
CPI_DRUG_WEEKLY.csv	External	Week	Macroeconomic and pricing context used to add broader environmental pressure signals.
FLUNET_CANADA_WEEKLY.csv	External	Week	Public-health and respiratory-demand context used to capture seasonal epidemiological pressure.
drug.txt/ther.txt/ingred.txt(+IA)	Reference	Drug / ingredient	Official molecule and therapeutic enrichment from Health Canada DPD used to standardize identity and ATC mapping.

5.3 External and Reference Data Sources

The external and reference sources below play different roles in the pipeline: some standardize product identity, while others add market or epidemiological context.

Source	Role Type	Business Value
Health Canada DPD	Reference enrichment	Standardizes molecule and ATC identity, improves item mapping quality, and supports downstream external-data alignment.
Google Trends	External demand-context signal	Provides ATC-level market-attention signals that can act as an early demand proxy.
Statistics Canada CPI	External macro-context signal	Adds broad pricing and supply-environment context through lagged CPI features.
WHO FluNet (Canada)	External epidemiological signal	Captures respiratory virus pressure that may affect demand for relevant therapeutic categories.

5.4 How External Data Is Integrated

The notebook integrates external and reference data in four different ways.

5.4.1 DPD

DPD is not primarily used as a forward-looking market signal. Instead, it acts as the standardization layer for:

- molecule enrichment,
- ATC mapping,
- and therapeutic-class alignment.

5.4.2 Google Trends

Google Trends is aggregated to an ATC-level weekly composite index. It is merged to the modeling panel using:

$$(\texttt{ATC_LEVEL1},\ \texttt{date_monday})$$

after a one-week lag so that only information available before the prediction week is used.

5.4.3 CPI

CPI features are transformed into lagged weekly macro context variables and merged by `date_monday`. They provide environmental context rather than item-specific signals.

5.4.4 FluNet

FluNet features are transformed into lagged weekly epidemiological variables and merged by `date_monday`. These features are particularly relevant for therapeutic categories affected by seasonal respiratory activity.

5.5 DPD Contribution in the Recorded Run

The current recorded run shows that DPD plays a major role in product standardization.

5.5.1 Molecule Source Breakdown

Molecule Source	Item Count
ACTIVEITEMS	30,440
DPD	23,911
PARSED	5,640
TRAIN_MAIN_UNIQUE	423

5.5.2 ATC Source Breakdown

ATC Source	Item Count
DPD_OFFICIAL	52,595
TFIDF_MATCH	2,153
KNOWLEDGE_BASE	987
UNMAPPED	4,679

Mapped ATC_LEVEL1 coverage in the recorded run is 55,735 items out of 60,414, or approximately 92.3%.

Key Takeaway

Internal product data is the primary source for molecule identity, while DPD is the primary standardized source for ATC mapping in the current implementation.

5.6 Key Business Definitions

Item
The operational product identifier used in internal systems.

DIN
Drug Identification Number. In the notebook it is normalized into DIN_KEY.

Final Molecule
The standardized molecule assigned after harmonization across internal and reference sources.

ATC Level 1
High-level therapeutic class used for grouping and external trend alignment.

ATC Level 3
More detailed therapeutic subgroup retained for enrichment and dimensional reporting.

Risk Score
The model-generated shortage risk score exported to dashboard tables.

5.7 Data Standardization Rules

The preprocessing logic applies the following business rules:

- **Final molecule priority**: train_P2 main name, then active-items molecule, then DPD molecule, then parsed molecule, else UNKNOWN.
- **DIN priority**: train_P2 map first, then active-items map.
- **ATC priority**: DPD official mapping first, then DPD-derived knowledge-base inference, then TF-IDF-assisted fallback.
- **Quality gate**: rows with invalid DIN_KEY or UNKNOWN FINAL_MOLECULE are removed.
- **Recorded run time window**: 2023-07-03 to 2023-12-25.

Chapter 6

Label Definition, Time Alignment, and Features

6.1 Prediction Target

The modeling target is `label_4w`. In business terms, the question is:

> Will this item experience a shortage within the next four weeks?

6.2 Label Definition

The notebook defines the label in a forward-looking way:

$$\texttt{label_4w}(i,t) = \begin{cases} 1, & \text{if item } i \text{ has a shortage in any week from } t+1 \text{ to } t+4 \\ 0, & \text{if the next four weeks are fully observed and no shortage occurs} \\ \text{NA}, & \text{if the future window is incomplete} \end{cases}$$

Key Takeaway

Recent observations are not forced into a 0 or 1 label if the future outcome window is incomplete. This avoids tail bias.

6.3 Prediction Timing Assumption

The implementation is aligned to the following operational assumption:

> Predictions are generated at the start of week t using information available up to the end of week $t-1$.

This matters because:

- current-week operational inputs are aligned to $t-1$,
- external weekly signals are lagged before merge,
- and the model is prevented from using future information.

6.4 Feature Engineering Summary

In the current recorded run, the feature-engineering stage produces:

- 245,622 item-week rows,
- 9,447 distinct active items in the panel window,

- a date range from 2023-07-03 to 2023-12-25,
- 121 engineered feature columns before final feature filtering,
- a pre-model positive rate of 0.2335 for `label_4w`.

6.5 Feature Families

The engineered features can be grouped into the families below. This framing is useful for explaining what kinds of risk signals the model is actually learning from.

Feature Family	Level	Business Purpose
Operational history	Item-week	Capture order, delivery, receiving, backlog, and other short-term operational instability patterns that often precede shortage outcomes.
Weekly sales	Item-week	Capture recent demand volume and volatility so the model can distinguish structurally low-volume items from items experiencing demand pressure.
Static item attributes	Item	Capture structural product characteristics such as dosage form, therapeutic grouping, and item-level product identity.
Vendor concentration	Item / DIN	Measure supplier diversification versus dependency so the model can detect concentration risk when too much volume sits with one source.
Lead time	Item / DIN	Measure logistics delay exposure and replenishment friction using weighted average lead-time signals.
External signals	Week / ATC-week	Add demand, macroeconomic, and epidemiological context that may explain broad market pressure beyond internal operations alone.
Lagged shortage history	Item-week	Capture persistence and recurrence patterns without leaking the future target window into the model.

6.6 Most Important Features in the Recorded Final Run

The final selected model in the current recorded run identifies the following as the most important features at the validation-stage model-importance view.

Feature	Importance	Interpretation
leadtime_wavg_item	704	Weighted item-level lead time exposure; higher values suggest slower replenishment cycles and less ability to respond quickly when demand or supply conditions shift.
vendor_top_share_din	602	Degree of supplier concentration at the DIN level; high values indicate that supply for therapeutically equivalent products may still be dominated by a small number of vendors.
vendor_top_share_item	582	Degree of supplier concentration at the item level; this helps identify items whose continuity is vulnerable to disruption at a single main source.
leadtime_wavg_din	506	DIN-level logistics latency exposure; useful when the risk is not just item-specific but shared across equivalent product groupings.
ATC_LEVEL1	359	Therapeutic-class effect; this feature helps the model learn that some therapeutic areas systematically behave differently from others under demand or supply stress.
QTY_ORD_mean_12w	357	Average recent internal ordered quantity; this reflects steady operational demand and replenishment scale over the recent 12-week window.
vendor_hhi_din	328	Supplier concentration risk at the DIN level using a fuller concentration measure than top-share alone, capturing how diversified or concentrated the source mix really is.
WS_QTY_ORD_std_12w	318	Weekly demand volatility over the recent 12-week window; spikes or unstable ordering behavior can be an early sign of market stress or fulfillment instability.

Chapter 7

Modeling Workflow

7.1 Temporal Split

The notebook uses a temporal split, not a random split. This is the correct design for an operational forecasting problem because it prevents future information from leaking into model training.

Split	Date Range	Rows	Purpose
Train	2023-07-03 to 2023-10-09	141,705	Fit model parameters and create rolling tuning folds using only historical information that would have been available at the time.
Validation	2023-10-16 to 2023-10-30	28,341	Optimize thresholds and support final model selection without touching the held-out test period.
Test	2023-11-06 to 2023-11-27	37,788	Final held-out out-of-sample evaluation used for client-facing performance reporting.

7.2 Benchmarks and Candidate Models

The notebook contains two different model groups.

7.2.1 Benchmark Models

These are used to establish reference performance:

- Majority class,
- Random baseline,
- Logistic Regression.

7.2.2 Tuned Candidate Models

These are the models eligible for final automated selection:

- XGBoost,
- LightGBM,
- Random Forest,
- CatBoost.

7.3 Hyperparameter Tuning

Hyperparameter tuning is performed in Section 7 using rolling folds created inside the training window. This is an important design choice because it avoids using the external validation period as the sole hyperparameter-tuning target.

7.4 Threshold Optimization

Section 8 evaluates three threshold strategies on the validation set:

- F1-maximizing threshold,
- recall-constrained threshold,
- precision-constrained threshold.

In the current recorded run, the deployed threshold is the F1-based threshold:

$$\texttt{FINAL_THRESHOLD} = 0.4543$$

7.5 Automated Model Selection Logic

The final model selection process is implemented as follows:

1. tune each candidate model in Section 7,
2. use the highest Section 7 rolling-fold mean PR-AUC as the primary selection anchor,
3. retrieve the validation threshold result from Section 8,
4. if another tuned candidate is within 0.05 validation F1 of the current best candidate, prefer the model with higher validation recall,
5. export the resulting model name and threshold as `FINAL_MODEL_NAME` and `FINAL_THRESHOLD`.

Key Takeaway

The current notebook does not simply pick the model with the best single validation number. It combines rolling-fold development scoring with a validation-stage operating-threshold comparison and a recall-favoring tolerance rule.

7.6 Recorded Final Test Performance

The held-out test set is the most appropriate client-facing performance reference for the current recorded run because it reflects data that was not used for model tuning or threshold selection.

Metric	Recorded Value
Test PR-AUC	0.5869
Test ROC-AUC	0.7817
Test Precision	0.5139
Test Recall	0.5915
Test F1	0.5500
TP	5,732
FP	5,421
FN	3,959
TN	22,676
Predicted positive rate	0.2951
Observed positive rate	0.2565

Table 7.1: Held-out test performance for the final selected model.

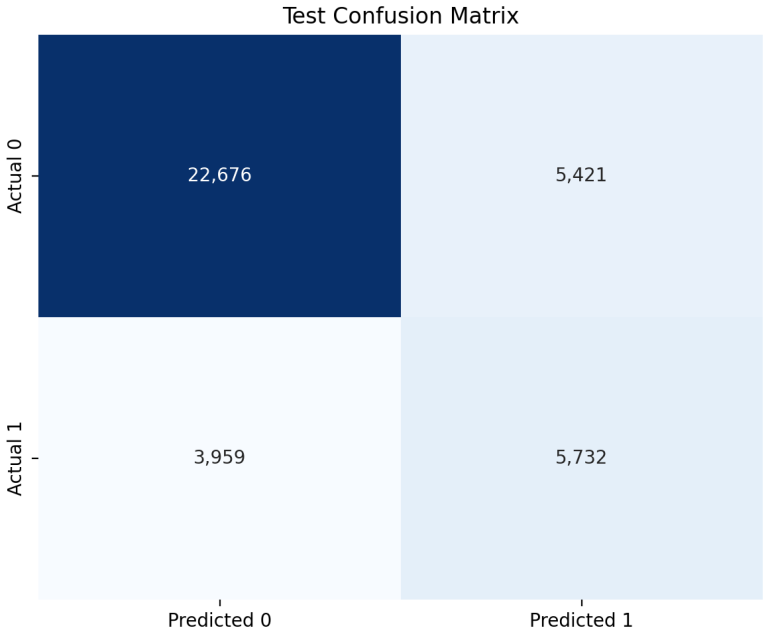

Figure 7.1: Test confusion matrix for the final selected model.

Interpretation. The confusion matrix shows that the model correctly flagged 5,732 shortage rows and missed 3,959, while also correctly retaining 22,676 non-shortage rows and generating 5,421 false positives. This means the model surfaces a substantial share of future shortage events, but flagged rows should still be reviewed before action is taken. The result is consistent with using the model as an early-warning and prioritization tool.

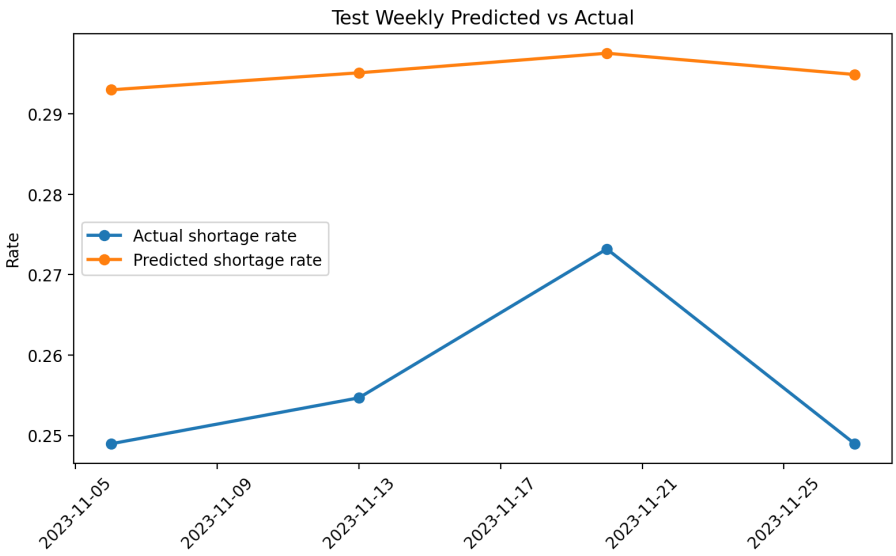

Figure 7.2: Weekly predicted versus observed shortage rate on the held-out test period.

Interpretation. The chart shows that the model tracks the broad week-to-week direction of shortage pressure across the held-out test period. Both series rise into the week of `2023-11-20` and then ease afterward, which indicates that the model captures the main temporal pattern in the test window. The predicted shortage rate remains above the observed rate in each week, so the model appears directionally useful but somewhat conservative in level.

Key Takeaway

The held-out test results show that the final model captures a meaningful share of future shortage events while maintaining a reasonable precision-recall balance on unseen data. The weekly comparison plot is especially useful for client readers because it shows whether predicted shortage intensity tracks observed shortage behavior over time rather than only performing well in aggregate.

Chapter 8

Interpretability and Explanation Outputs

8.1 Interpretability Layers

The notebook provides three different interpretability layers:

1. **Model-native feature importance**: a fast global view of which features matter most.
2. **Global SHAP summaries**: richer direction-aware global explanation plots.
3. **Row-level reasons**: top feature-level explanation records exported to Azure SQL.

8.2 Validation-Stage Interpretability

After final model selection in Section 9, the notebook attempts to create a validation-stage global SHAP summary using a validation sample. This output is saved as:

- `Output_v2/section9_shap_summary.png`

8.3 Test-Stage Interpretability

Section 10 includes optional test-stage SHAP outputs for the final selected model. In the current recorded run, the notebook saves:

- `Output_v2/test_evaluation/test_shap_importance.csv`,
- `Output_v2/test_evaluation/test_shap_importance_bar.png`,
- `Output_v2/test_evaluation/test_shap_summary_beeswarm.png`.

These outputs are created when the selected model is compatible with the current SHAP workflow. In the current implementation, this is expected for the tuned tree-based candidate set.

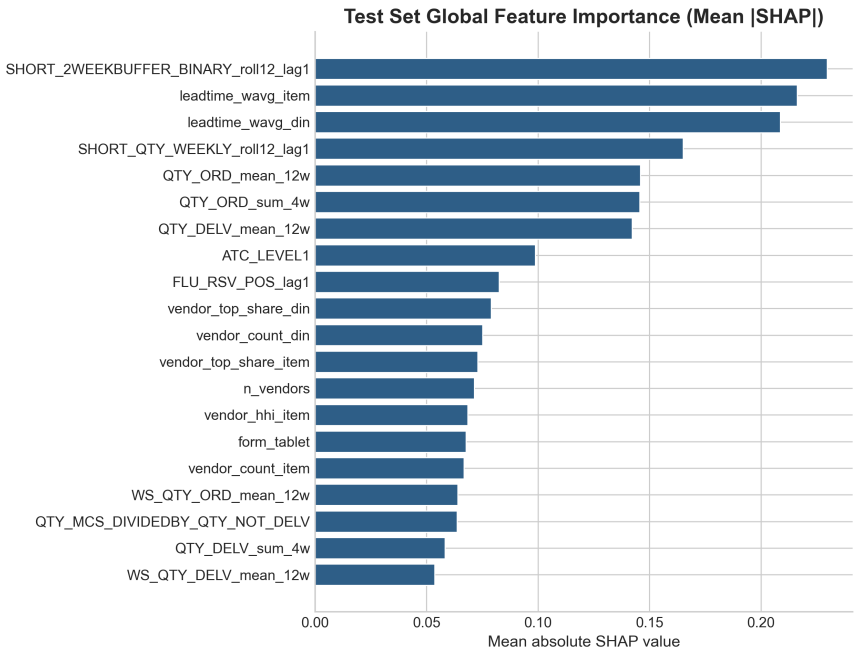

Figure 8.1: Test-set global feature importance based on mean absolute SHAP value.

Interpretation. This chart identifies which features have the largest average influence on predictions across the held-out test set. The strongest drivers are concentrated around lead time, vendor concentration, therapeutic class, and recent demand behavior. That pattern supports the business view that shortage risk is shaped by a combination of replenishment friction, supplier dependency, and operational pressure.

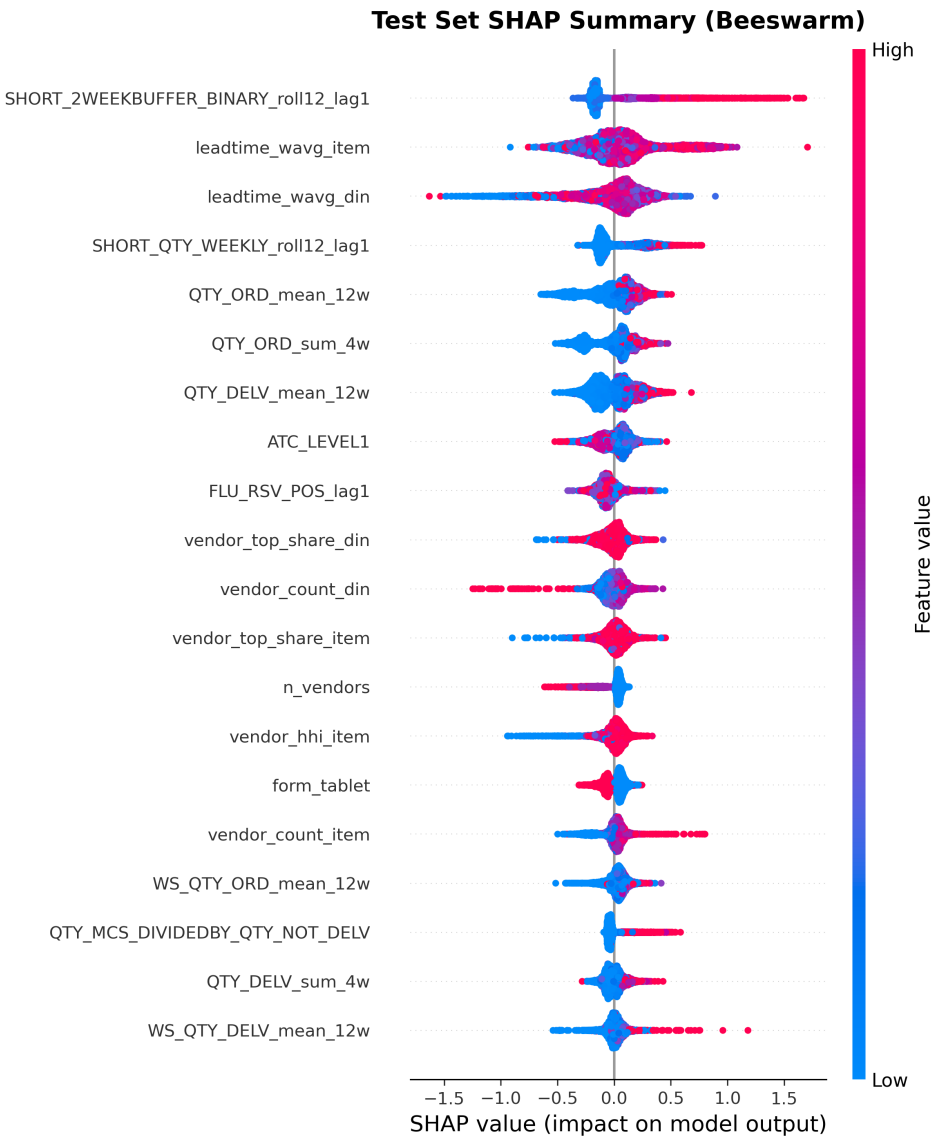

Figure 8.2: Test-set SHAP summary beeswarm plot for the final selected model.

Interpretation. The beeswarm plot shows both feature importance and direction, making it possible to see whether higher or lower feature values tend to increase predicted shortage risk. This helps check whether the model's behavior is broadly consistent with business logic rather than relying only on a ranked importance list. It is also useful for spotting features whose effects vary across different item-week cases.

8.4 Row-Level Explanation Export

Section 11 can export row-level explanation factors to Azure SQL through `dbo.fact_risk_reason`. In the current configuration:

- reasons are enabled,
- only predicted positives are considered,
- only the last 4 weeks are considered,
- at most 2,000 rows are exported,
- and the top 3 SHAP drivers per row are retained.

Interpretability outputs describe model behavior. They support review and prioritization, but they should not be treated as causal proof.

Chapter 9

Deployment and Azure SQL Export

9.1 Deployment Target

The current implementation exports dashboard outputs to Azure SQL under the dbo schema.

9.2 Runtime and Environment Requirements

To run the notebook successfully, the environment must satisfy the following:

- the current working directory must contain the `Internaldata` folder,
- the required Python dependencies must be installed,
- the selected model libraries must be available,
- and the SQL client environment must have `ODBC Driver 18 for SQL Server` installed.

9.3 Execution Order

The notebook should be run in the following order:

1. external data acquisition,
2. data loading and preprocessing,
3. feature engineering,
4. modeling data preparation and validation checks,
5. benchmark and candidate model training,
6. hyperparameter tuning,
7. threshold optimization,
8. final model selection and interpretability,
9. final test evaluation,
10. Azure SQL export.

9.4 Default Deployment Logic

The Azure export logic reuses the final selected model and threshold by default:

- if `FINAL_MODEL_NAME` exists, export uses it,
- if `FINAL_THRESHOLD` exists, export uses it,
- if no threshold is stored, export falls back to `0.5`,
- manual override is possible only when a specific `MODEL_NAME` or `THRESHOLD` is supplied.

9.5 Export Behavior

The current Azure SQL export logic includes:

- a 26-week history window,
- configurable export mode (`incremental`, `rerun_window`, or `full_refresh`),
- full refresh of dimension tables,
- append-only registry and performance tracking,
- item-week risk-score export,
- optional SHAP reason export,
- predicted-score display logic,
- and dedicated model-performance summary and weekly monitoring tables.

9.6 Output Tables

The export layer produces the tables below. Together they support dashboard drill-down, scoring, explanation, and performance monitoring.

Table	Level	Purpose
`dim_time`	Time	Calendar dimension for weekly reporting, filtering, and consistent time-based dashboard slicing.
`dim_molecule`	Molecule	Molecule and therapeutic lookup dimension used to support business-readable drill-down above the item level.
`dim_item`	Item	Item master for readable dashboard drill-down, product descriptions, and stable item-level context.
`model_registry`	Model instance	Stores the selected model identity and its recorded evaluation metrics so dashboard consumers can see which model version produced a run.
`run_registry`	Export run	Stores export-run metadata, timing, and audit trail information needed for traceability and operational support.
`fact_risk_score`	Item-week	Stores the core scoring output: risk score, risk bucket, predicted shortage flag, and observed outcome when available.
`fact_risk_reason`	Item-week reason	Stores top row-level explanation factors for selected predictions so high-risk rows can be interpreted rather than only ranked.
`model_performance_summary`	Run	Stores final test summary metrics for KPI cards and headline model-performance monitoring.
`model_performance_weekly`	Run-week	Stores weekly test performance and predicted-versus-actual monitoring values to show whether performance remains stable across weeks.

9.7 What the Client Uses Most Often

For business-facing dashboard use, the most important table is usually `fact_risk_score`. For model-performance monitoring, the most important tables are:

- `model_performance_summary`,
- `model_performance_weekly`,
- and, when explanations are enabled, `fact_risk_reason`.

Before the notebook is published to GitHub or reused in a broader deployment context, Azure SQL credentials must be removed from notebook configuration and replaced with secure configuration management.

Part III

Client Guidance and Reference

Chapter 10

How to Use the Results

10.1 How to Read the Risk Score

The risk score is a model-generated risk signal. A higher value means the model sees stronger evidence that the item may enter shortage within the next four weeks.

10.2 How to Read the Risk Bucket

The current dashboard uses three score buckets:

- **LOW**: score below 0.33,
- **MEDIUM**: score from 0.33 up to but not including 0.66,
- **HIGH**: score of 0.66 or above.

10.3 How to Read Predicted and Actual Shortage

- **Predicted shortage**: the model's binary flag after applying the selected threshold.
- **Actual shortage**: the observed outcome for the target horizon, when available.
- **Actual available**: indicates whether the observed target is currently known.

10.4 How to Use Performance Tables

`model_performance_summary` should be used for:

- KPI cards,
- confusion-matrix reporting,
- and overall final-test model monitoring.

`model_performance_weekly` should be used for:

- weekly precision, recall, and F1 trends,
- predicted-versus-actual shortage-rate charts,
- and weekly monitoring of final-test behavior.

10.5 How to Use Reasons

The explanation table helps answer:

- why a row received a high score,
- which signals contributed most strongly,
- and which items should be reviewed first.

Key Takeaway

The strongest operational use of the system is triage. It helps determine what deserves attention first; it does not replace business review.

Chapter 11

Risks, Assumptions, and Limitations

11.1 Operational Assumptions

The notebook assumes:

- source files are available and structurally stable,
- item-to-DIN and item-to-molecule mappings are sufficiently reliable,
- weekly alignment across internal and external sources is valid,
- the current shortage definition remains business-relevant,
- and the notebook is run from a working directory containing `Internaldata`.

11.2 Key Risks

- source latency can reduce timeliness,
- schema drift can break joins or features,
- recent labels are incomplete by design,
- external signal behavior can change over time,
- and future runs can auto-select a different final model.

11.3 Known Limitations

- this is a notebook implementation, not a fully packaged production service,
- explanations describe model behavior, not causality,
- risk scores are decision-support signals rather than stand-alone decisions,
- and the current deployment path is coupled to the Azure SQL schema and table design defined in Section 11.

Chapter A

Appendix A: Azure SQL Output Dictionary

This appendix documents the Azure SQL outputs created by the current notebook. The table definitions below are aligned to the current Section 11 implementation.

A.1 dbo.dim_time

Purpose. Calendar dimension for weekly reporting.

Grain. One row per `date_monday`.

Column	SQL Type	Definition
`date_monday`	DATE	Weekly reporting date used as the central time key for dashboard slicing, joining, and time-based monitoring.
`year`	INT	Calendar year derived from `date_monday`.
`month`	INT	Calendar month derived from `date_monday`.
`week`	INT	ISO week number derived from `date_monday`.
`quarter`	INT	Calendar quarter derived from `date_monday`.
`yearweek`	INT	Compact year-week code for sorting and grouping in weekly dashboard views.

A.2 dbo.dim_molecule

Purpose. Molecule lookup dimension.

Grain. One row per `FINAL_MOLECULE`.

Column	SQL Type	Definition
`FINAL_MOLECULE`	NVARCHAR(255)	Standardized molecule name used for molecule-level reporting and grouping.
`ATC_LEVEL1`	NVARCHAR(8)	High-level therapeutic class associated with the molecule.
`ATC_LEVEL3`	NVARCHAR(16)	More detailed therapeutic subgroup associated with the molecule.

A.3 dbo.dim_item

Purpose. Item master dimension for dashboard drill-down.

Grain. One row per `ITEM_NUM_CLEAN`.

Column	SQL Type	Definition
ITEM_NUM_CLEAN	BIGINT	Cleaned item identifier used as the main dashboard drill-down key.
DIN_KEY	NVARCHAR(32)	Normalized DIN linked to the item after source harmonization.
FINAL_MOLECULE	NVARCHAR(255)	Standardized molecule associated with the item.
ATC_LEVEL1	NVARCHAR(8)	High-level therapeutic class for the item.
ATC_LEVEL3	NVARCHAR(16)	Detailed therapeutic subgroup for the item.
item_class_group	NVARCHAR(64)	Business class group used for readable segmentation in the dashboard.
EN_FORM	NVARCHAR(64)	Dosage or physical form of the product.
ITEM_EN_DESC	NVARCHAR(255)	Full English description used in detailed display views.
SHORT_EN_DESC	NVARCHAR(255)	Short English description used in compact dashboard views.

A.4 dbo.model_registry

Purpose. Registry of exported model instances.

Grain. One row per `model_id`.

Column	SQL Type	Definition
model_id	NVARCHAR(64)	Unique exported model identifier.
model_name	NVARCHAR(64)	Selected model family name.
pr_auc	FLOAT	Final test PR-AUC recorded for the model at export time.
roc_auc	FLOAT	Final test ROC-AUC recorded for the model at export time.
precision	FLOAT	Final test precision at the selected threshold.
recall	FLOAT	Final test recall at the selected threshold.
f1	FLOAT	Final test F1 at the selected threshold.
tp	INT	Final test true positives.
fp	INT	Final test false positives.
fn	INT	Final test false negatives.
tn	INT	Final test true negatives.
threshold	FLOAT	Threshold used in the export run.
test_start	DATE	Earliest test week.
test_end	DATE	Latest test week.
test_rows	INT	Number of rows in the held-out test set.
created_at	DATETIME2	Timestamp when the registry row was written.
notes	NVARCHAR(255)	Free-text note written by the current notebook to describe the model-selection context.

A.5 dbo.run_registry

Purpose. Registry of Azure SQL export executions.

Grain. One row per `run_id`.

Column	SQL Type	Definition
run_id	NVARCHAR(64)	Unique export-run identifier.
model_id	NVARCHAR(64)	Model version used during the run.
model_name	NVARCHAR(64)	Selected model family name.

Column	SQL Type	Definition
target_col	NVARCHAR(32)	Target column name; current value is label_4w.
threshold	FLOAT	Threshold used for predicted_shortage.
display_rule	NVARCHAR(32)	Rule controlling risk_score_display.
run_ts	DATETIME2	Timestamp when the run was written.
data_start	DATE	Earliest exported scoring week.
data_end	DATE	Latest exported scoring week.
test_start	DATE	Earliest held-out test week.
test_end	DATE	Latest held-out test week.
n_rows	INT	Number of exported scoring rows.
is_incremental	SMALLINT	Indicator of whether the export reused an existing history state.
scoring_export_mode	NVARCHAR(32)	Export mode; for example rerun_window.

A.6 dbo.fact_risk_score

Purpose. Main item-week risk output fact table.

Grain. One row per run_id + ITEM_NUM_CLEAN + date_monday.

Column	SQL Type	Definition
run_id	NVARCHAR(64)	Export-run identifier.
model_id	NVARCHAR(64)	Model version used for scoring.
ITEM_NUM_CLEAN	BIGINT	Cleaned item identifier.
FINAL_MOLECULE	NVARCHAR(255)	Molecule associated with the scored row.
date_monday	DATE	Weekly key for the scored row.
risk_score	FLOAT	Model-generated shortage risk score, where higher values represent greater predicted shortage risk.
risk_bucket	NVARCHAR(16)	Dashboard risk bucket derived from the raw score for easier business-facing triage.
risk_score_display	FLOAT	Display-ready score after dashboard display logic has been applied.
predicted_shortage	SMALLINT	Binary prediction created from score and threshold.
actual_shortage	FLOAT	Observed target value when available.
actual_available	SMALLINT	Indicator that the observed target exists for the row's horizon.

A.7 dbo.fact_risk_reason

Purpose. Row-level explanation output table.

Grain. Up to REASON_TOPK rows per run_id + ITEM_NUM_CLEAN + date_monday.

Column	SQL Type	Definition
run_id	NVARCHAR(64)	Export-run identifier.
ITEM_NUM_CLEAN	BIGINT	Explained item identifier.
FINAL_MOLECULE	NVARCHAR(255)	Molecule associated with the explained row.
date_monday	DATE	Weekly key of the explained row.
rank_	INT	Rank among the retained explanation factors for the row.
feature_name	NVARCHAR(128)	Name of the explanatory feature retained as a top driver for the row.

Column	SQL Type	Definition
contribution	FLOAT	Signed SHAP contribution for the row; positive values push predicted risk upward and negative values push it downward.
feature_value	FLOAT	Numeric feature value when conversion succeeds.

A.8 dbo.model_performance_summary

Purpose. Final test summary table for model-performance dashboarding.

Grain. One row per `run_id`.

Column	SQL Type	Definition
run_id	NVARCHAR(64)	Export-run identifier and primary key.
model_id	NVARCHAR(64)	Model version used during the run.
model_name	NVARCHAR(64)	Selected model family name.
threshold	FLOAT	Threshold used for final test classification.
test_start	DATE	Earliest test week.
test_end	DATE	Latest test week.
test_rows	INT	Number of rows in the held-out test set.
pr_auc	FLOAT	Final test PR-AUC.
roc_auc	FLOAT	Final test ROC-AUC.
precision	FLOAT	Final test precision.
recall	FLOAT	Final test recall.
f1	FLOAT	Final test F1.
tp	INT	Final test true positives.
fp	INT	Final test false positives.
fn	INT	Final test false negatives.
tn	INT	Final test true negatives.
created_at	DATETIME2	Timestamp when the summary row was written.

A.9 dbo.model_performance_weekly

Purpose. Weekly final-test monitoring table for performance dashboards.

Grain. One row per `run_id + split_type + date_monday`.

Column	SQL Type	Definition
run_id	NVARCHAR(64)	Export-run identifier.
model_id	NVARCHAR(64)	Model version used during the run.
model_name	NVARCHAR(64)	Selected model family name.
split_type	NVARCHAR(16)	Split label; current export writes `TEST`.
date_monday	DATE	Weekly time key.
rows	INT	Number of test rows in that week.
actual_rate	FLOAT	Observed shortage rate in that week.
predicted_rate	FLOAT	Predicted shortage rate in that week.
avg_score	FLOAT	Mean model score in that week.
precision	FLOAT	Weekly precision.
recall	FLOAT	Weekly recall.

Column	SQL Type	Definition
`f1`	FLOAT	Weekly F1.
`tp`	INT	Weekly true positives.
`fp`	INT	Weekly false positives.
`fn`	INT	Weekly false negatives.
`tn`	INT	Weekly true negatives.
`created_at`	DATETIME2	Timestamp when the row was written.

Chapter B

Appendix B: Additional Test Diagnostics

The figures in this appendix are technical diagnostics from the held-out test set. They are useful for model review and technical maintenance, but they are not as essential as the confusion matrix and weekly predicted-versus-actual comparison in the main body of the report.

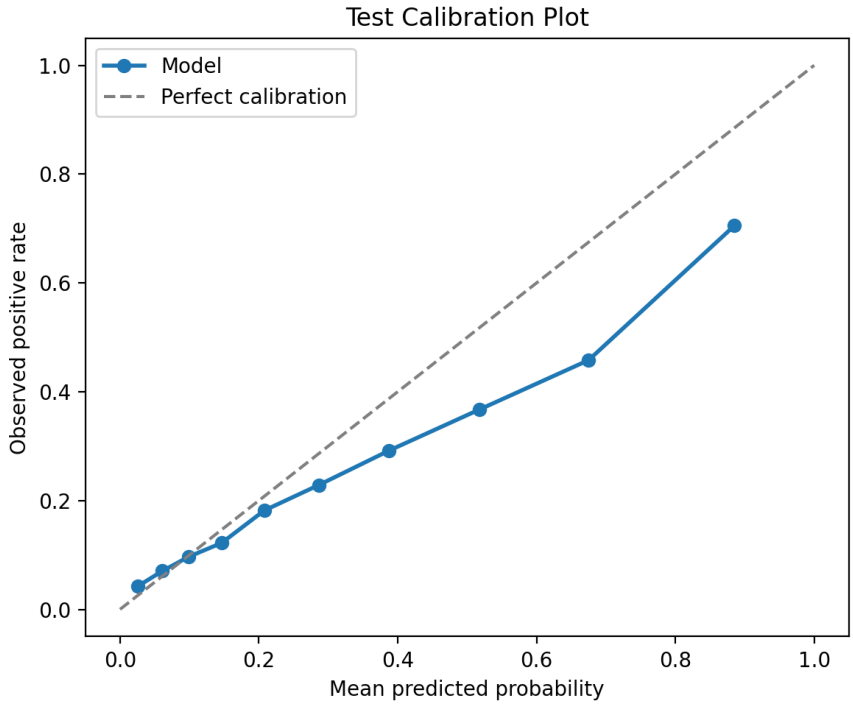

Figure B.1: Held-out test calibration plot.

Interpretation. The calibration plot shows whether predicted probabilities align with observed shortage rates. Points close to the diagonal indicate good calibration, while systematic deviation indicates that the score scale is too optimistic or too conservative. This is especially useful here because the weekly comparison suggests that predicted shortage intensity may be running above observed levels.

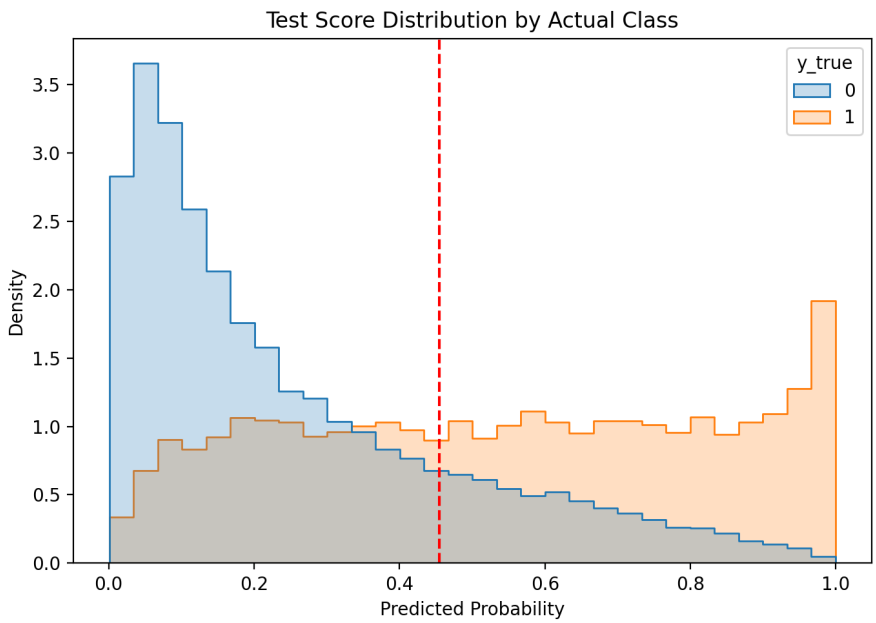

Figure B.2: Held-out test score distribution by actual class.

Interpretation. The score-distribution plot shows how well the model separates shortage rows from non-shortage rows on the held-out test set. Greater separation between the two distributions indicates stronger ranking ability, while overlap helps explain why false positives and false negatives remain after threshold selection. This makes the plot useful for judging whether performance limits come from class overlap or from threshold choice.

Chapter C

Appendix C: Glossary

PR-AUC
Area under the precision-recall curve. In this solution it is the primary development-stage model comparison metric.

Threshold
Decision cutoff used to convert a continuous score into a binary shortage prediction.

Auto-selection
Notebook logic that chooses the final model from the tuned candidate set based on the implemented model-selection rule.

SHAP
Feature-contribution framework used for global and row-level explanation.

Item-week
The main modeling grain used by the notebook.

ATC
Anatomical Therapeutic Chemical classification used for therapeutic grouping.

Retention window
Number of historical weeks retained in exported dashboard tables.